



Nest & Co.

Hotel Chatbot — Working Documentation

Powered by Flask + Groq (LLaMA 3.3 70B)

Assistant
Cleo

Stack
Flask + Groq

1. Project Overview

Nest & Co. is a Gen Z–styled hotel chatbot web app. It replaces the typical stiff hotel assistant with Cleo — a warm, casual, and relatable virtual concierge. The frontend runs in the browser and talks to a Flask backend which calls the Groq API to power AI responses.

What it does

- Answers guest questions about rooms, pricing, amenities, bookings, and policies
- Maintains conversation history for context-aware replies
- Provides quick-action buttons for common queries
- Streams responses from LLaMA 3.3 70B via Groq's free API

Tech Stack

Layer	Technology	Purpose
Frontend	HTML / CSS / Vanilla JS	Chat UI, animations, quick prompts
Backend	Python Flask	API server, routing
AI Model	LLaMA 3.3 70B (Groq)	Natural language responses
Fonts	Playfair Display, DM Sans, Caveat	Cozy Gen Z aesthetic

2. File Structure

The project is intentionally minimal — two files is all you need to run it:

```
your-project/  
├─ app.py           ← Flask backend + Groq API calls  
└─ templates/  
    └─ index.html   ← Full chat UI (HTML + CSS + JS)
```

💡 Why templates/?

Flask's `render_template()` looks for HTML files inside a `templates/` folder by default. Make sure `index.html` lives there.

3. Setup & Installation

Step 1 — Install Python dependencies

You need Flask and the Groq Python SDK. Run this in your terminal:

```
pip install flask groq
```

Step 2 — Project folder setup

Create the folder structure and place your files:

```
mkdir my-hotel-bot
cd my-hotel-bot
mkdir templates
# place app.py here
# place index.html inside templates/
```

Step 3 — Run the server

```
python app.py
```

Then open your browser and go to: <http://localhost:5000>

⚠ API Key Notice

The Groq API key is currently hardcoded in app.py. For production use, move it to an environment variable: `os.environ.get('GROQ_API_KEY')`

4. How It Works

Request Flow

#	Who	What happens
1	Browser (JS)	User types a message → JS appends it to history array
2	Browser → Flask	POST /chat with full message history as JSON
3	Flask (app.py)	Prepends SYSTEM_PROMPT to messages, calls Groq API
4	Groq → Flask	LLaMA 3.3 70B generates Cleo's response
5	Flask → Browser	Returns response as JSON {response: '...'}
6	Browser (JS)	Adds bot message to chat window with thought tag

Conversation Memory

The frontend stores all messages in a JavaScript array called `history[]`. Every time the user sends a message, the entire history is sent to Flask — this gives Cleo full context of the conversation without any server-side session storage.

System Prompt

Cleo's personality, hotel info, and tone rules are all defined in `SYSTEM_PROMPT` inside `app.py`. This is injected as the first message (role: system) in every API call, so Cleo always knows who she is and where she works.

5. Customization Guide

Changing the Hotel Name

Update in two places:

- `app.py` → `SYSTEM_PROMPT`: change all instances of "Nest & Co." to your hotel name
- `templates/index.html` → Find "Nest & Co." in the HTML and JS greeting

Changing the Assistant Name

Currently "Cleo" — update in three places:

- `app.py` → `SYSTEM_PROMPT`: first line "You are Cleo"
- `index.html` → status pill text: "Cleo is online "
- `index.html` → JS: `isBot ? 'Cleo' : 'You'`

Updating Room Prices / Amenities

All hotel info lives in the `SYSTEM_PROMPT` block in `app.py`. Edit the `ROOM TYPES`, `AMENITIES`, and `BOOKING` sections to match your actual hotel.

Changing Cleo's Tone

The `TONE EXAMPLES` section in `SYSTEM_PROMPT` controls how Cleo speaks. Add more examples to reinforce the style you want. The model mimics the pattern of examples closely.

Changing the AI Model

In `app.py`, find the model parameter and swap it:

Model	Speed vs Quality
<code>llama-3.3-70b-versatile</code>	Best quality — current default
<code>llama-3.1-8b-instant</code>	Faster, lighter — good for testing
<code>mixtral-8x7b-32768</code>	Long context, good for detail

6. Troubleshooting

Problem	Fix
Cleo uses wrong hotel name	Update SYSTEM_PROMPT in app.py. The model falls back to training data if the prompt is unclear — be explicit with NEVER rules.
'oh no, something glitched' error	Check terminal for Python errors. Usually a bad API key or network issue.
Template not found error	Make sure index.html is inside a templates/ folder, not the root directory.
ModuleNotFoundError: flask	Run: pip install flask groq
Port already in use	Change port in app.py: app.run(port=5001) or kill the process using port 5000.
Responses sound too formal	Add more casual tone examples to SYSTEM_PROMPT. The model follows examples closely.

7. API Reference

POST /chat

The only API endpoint. Receives conversation history, returns Cleo's reply.

Field	Type	Description
messages	Array	Full conversation history [{role, content}]
role	String	"user" or "assistant"
content	String	The message text

Request Example

```
POST /chat
Content-Type: application/json

{
  "messages": [
    {"role": "user", "content": "do you allow pets?"}
  ]
}
```

Response Example

```
{
  "response": "yeah we love pets at Nest & Co.! small pets
are totally welcome, just a $30/night fee "
}
```

